v1.0

HOW RELIABLE ARE THE LOGS?

Kernel-User Telemetry Desynchronization

Detection Engineering Perspective

FOCUS AREA	DETECTION TYPE	PLATFORM
Process Telemetry	**Kernel-User Desync**	**Windows ETW**

Birkan KES

February 2026

RESEARCH ACTIVE

Executive Summary

Modern security monitoring systems largely base their decisions about processes on log data. However, this data does not always represent the same point in time or the same reality. In Windows environments, there can be significant differences between the information recorded by the kernel at process creation time and the data read by user-mode tools while the process is running.

This difference creates a blind spot that may cause security tools to interpret processes incorrectly or incompletely. Detection approaches that rely heavily on process trees and command-line information can be misled, especially when they depend on user-mode data that can be manipulated by attackers.

Conclusion: Security logs may present two different realities that are internally consistent but contradictory to each other. What the kernel sees and what security tools display do not have to be the same.

This research demonstrates how this inconsistency in process creation telemetry can be exploited in practice and why detection strategies based on a single data source are insufficient. The findings show that protecting kernel-based data and correlating multiple telemetry sources are critical to improving detection reliability.

Introduction

In cybersecurity, many of us share a common habit: we look at logs and try to understand what happened in the system from there.

- **Is a process benign or suspicious?**
- **With which command line was this binary actually launched?**
- **Why was an alert triggered, or why was it not triggered?**

Most of the time, we answer these questions by analyzing telemetry collected through Sysmon, EDR, and SIEM. This approach is based on an assumption: logs accurately and completely reflect what truly happened in the system. In practice, however, this assumption is not always correct. Especially in the Windows architecture, different telemetry sources related to the same event are generated at different times and at different layers. As a result, logs do not always tell a single, consistent story.

Previous analyses of ETW (Event Tracing for Windows) clearly show that telemetry generated at the kernel and user-mode levels can sometimes diverge. In techniques such as parent process spoofing, meaningful differences may appear between what the kernel knows about a process and what user-mode tools present. This difference is less about the event itself and more about how the event is observed.

Does this inconsistency only affect the process tree, or should everything we see about the process itself be questioned?

At this point, the question naturally evolves into the following:

- **Which logs are truly reliable?**
- **Which ones only represent a momentary view and are open to manipulation?**

Process Environment Block (PEB) manipulation is just one example of a broader problem around this question. The core issue is the difference between the information recorded by the kernel at the moment of process creation and the user-mode data that security tools rely on while the process is running. Some fields recorded by the kernel during process creation cannot be changed after the event, while certain information read from user-mode memory during execution can be modified later with sufficient access.

This article does not aim to describe a single technique; rather, it aims to question how much and in what context we should trust logs. It focuses on a shared question for SOC analysts, detection engineers, red teams, and anyone interested in the internals of Windows:

At which layer does the truth reside, and which logs do we use to interpret it?

The Distinction Between Kernel and User-Mode Telemetry

Telemetry related to the same process may originate from different layers, depending on its source and the time of observation. The difference here is not about accuracy, but about context: some logs reflect the moment of process creation, while others represent the state of the process during runtime. When this distinction is not considered, telemetry can easily be misinterpreted

Critical question: If some logs represent the moment of process creation and others reflect the process during execution, which one are we using as a reference in detection and analysis?

Quick Comparison

Telemetry Source	Observation Layer	Time Context	Reliability Perspective
Sysmon Event ID 1	Kernel destekli (hybrid)	Creation time	Medium–High
User-mode Tools	User-mode	Runtime	Low–Medium
EDR Solutions	Hybrid (product-dependent)	Creation + Runtime	Variable

Detailed Analysis

□ Sysmon Event ID 1

- **Layer:** Kernel-supported (hybrid)
- **Time:** Process creation time
- **Trigger:** Starts with a kernel callback; some fields are enriched using user-mode sources.
- **Strength:** Captures the very early stage of process creation and is not affected by changes made after creation.
- **Limitation**: Metadata changes made after the creation moment are not visible; in short timing windows, kernel–user inconsistencies may occur.

□ User-mode Tools (Process Explorer, Task Manager)

- **Layer:** User-mode
- **Time:** Runtime
- **Trigger:** Reads metadata from the PEB while the process is running.
- **Strength:** Reflects the current state and allows real-time inspection.
- **Limitation**: The data is read from modifiable memory structures and is open to manipulation.

□ EDR Solutions

- **Layer:** Hybrid (Kernel + User)
- **Time:** Creation + Runtime (continuous monitoring)
- **Trigger:** Process relationships are monitored through kernel callbacks, ETW providers, and additional sensors.
- **Strength:** Provides an earlier and more robust observation point compared to pure user-mode enrichment
- **Limitation**: The UI layer may rely on user-mode data, and the depth of correlation depends on the product architecture.

The Process Environment Block (PEB) and Its Critical Role in Security Telemetry?

The Process Environment Block (PEB) is a data structure associated with every process running on Windows and resides entirely in user-mode address space. It stores critical metadata about how a process was started, which parameters it was launched with, and details about its execution environment. A large portion of security tools read process-related information directly or indirectly from the PEB.

The main fields within the PEB include:

Field	Description
CommandLine	The command line used to launch the process
ImagePathName	The full path of the executed binary
Environment	The process environment variables
Ldr	Loader structures that maintain the loaded modules
ProcessParameters	The structure that contains the process startup parameters

⚠ Critical Point: The PEB is not a read-only structure enforced by the kernel. It resides in user-mode address space and can be read and modified by a process with sufficient privileges.

In practice, this distinction leads to the following::

- The kernel sees the process startup conditions as a fixed snapshot.
- User-mode tools rely on runtime metadata that can be modified by an attacker.
- As a result, the same process may be perceived as different “realities” across different telemetry sources.

□ **Technical Explanation:** PEB manipulation does not change the code that the process actually executes. What is altered is the process metadata that security tools and analysts rely on. For this reason, PEB manipulation is not an execution technique; it is a deception technique that disrupts observability.

Kernel–User Telemetry and the Timing Window

What enables PEB manipulation is not a complex exploit chain, but a natural timing difference inherent in the Windows process creation model. During process creation, kernel and user-mode components are involved at different stages, which leads to telemetry related to the same process being generated at different times.

This flow can be summarized as follows:

Step	What Happens	State
T-0	The kernel creates the process; the image path, command line, and parent PID are defined	✓ Fixed
T-1	Kernel callbacks and ETW events are triggered; telemetry closest to the creation moment is generated	✓ Fixed
T-2	User-mode initialization begins; the PEB is populated and threads start	⚠ Variable
T-3	User-mode tools read data from the PEB; the process becomes observable	⚠ Manipulable

□ Critical Timing Window

Kernel telemetry is generated → Sysmon Event ID 1 is logged → However, user-mode enrichment may not yet have occurred or may take place at a different time. Within this short but sufficient window, an attacker can manipulate specific fields inside the PEB.

Conclusion: Kernel-based logs represent the state of the process at the moment of creation, while user-mode tools reflect the state observed after the process has started running. This is not a race condition; it is a divergence over time between two different observation points. **The logs are not incorrect**, they simply report the same event from different layers and at different points in time.

Telemetry Matrix: What Point in Time Does It Represent?

The central question of this study is: “The information I see about a process which point in time does it actually belong to?”

Logs are often approached from a true/false perspective. However, when it comes to Windows telemetry, the key factor is not correctness, but timing and the layer at which the observation is made. **Telemetry is inherently contextual**: without understanding when and where it was generated, even accurate data can lead to incorrect conclusions..

Source	Layer	Observation Time	Represented State	Manipulation Risk	Reliability
Kernel ETW (ProcessStart)	Kernel	Creation time	Kernel creation data (PID/PPID)	Low	★★★★★
PsSetCreateProcessNotifyRoutineEx	Kernel	Creation time	Raw EPROCESS callback	Very Low	★★★★★
Sysmon Event ID 1	Kernel → User	Creation time	Creation snapshot (cmdline/parent)	Medium	★★★★
EDR Process Tree (UI)	Hybrid	Post-creation	Enriched process view	Medium	★★★
Process Explorer / Task Manager	User-mode	Runtime	Live PEB state	High	★★
Memory Forensics	Offline	Acquisition time	Captured memory structures	Variable	★★★★
Sysmon Event ID 10	Kernel	Runtime	Process handle access	Low	★★★★

Not: This matrix was prepared by consolidating findings from technical documentation, open-source research, blog analyses, and controlled laboratory testing.The reliability level assigned to Sysmon Event ID 1 reflects the timing and configuration dependencies between the kernel callback trigger and user-mode enrichment..

2. The Structure of the Process Environment Block (PEB)

```
┌──────────────────────────────────────────────────────────────────────┐
│                        THREAD (fs/gs register)                       │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  TEB (Thread Environment Block)                              │   │
│  │  Offset: fs:[0x30] (x86) / gs:[0x60] (x64)                   │   │
│  │                                                              │   │
│  │   ┌──────────────────────┐                                   │   │
│  │   │ +0x000  ExceptionList │                                   │   │
│  │   │ +0x008  StackBase     │                                   │   │
│  │   │ +0x010  StackLimit    │                                   │   │
│  │   │ +0x030  PEB ──────────┼──┐  ←□ Pointer to PEB            │   │
│  │   └──────────────────────┘  │                                 │   │
│  └──────────────────────────────┼─────────────────────────────────┘   │
└──────────────────────────────────┼──────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────┐
│              PEB (Process Environment Block)                         │
│              Base Address: 0x00007FF...                              │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ +0x000  BYTE Reserved1[2]                                    │   │
│  │ +0x002  BYTE BeingDebugged       ⚠ Anti-debug flag           │   │
│  │ +0x003  BYTE Reserved2[1]                                    │   │
│  │ +0x008  PVOID Reserved3[2]                                   │   │
│  │ +0x018  PPEB_LDR_DATA Ldr ──────────────┐                    │   │
│  │ +0x020  PRTL_USER_PROCESS_PARAMETERS    │                    │   │
│  │         ProcessParameters ──────────┐   │                    │   │
│  │ +0x010  ImageBaseAddress            │   │                    │   │
│  └──────────────────────────────────────┼───┼────────────────────┘   │
└──────────────────────────────────────────┼───┼──────────────────────────┘
                                           │   │
                                           ▼   ▼
┌──────────────────────────────────────────┐   ┌──────────────────────────┐
│  RTL_USER_PROCESS_PARAMETERS             │   │  PEB_LDR_DATA            │
│  ⋆ Attacker Objective!                   │   │  (Loader Data)           │
│                                          │   │                          │
│ +0x000 MaximumLength                     │   │ +0x00 Length             │
│ +0x004 Length                            │   │ +0x04 Initialized        │
│ +0x008 Flags                             │   │ +0x08 SsHandle           │
│ +0x010 Environment                       │   │ +0x10 InLoadOrderModuleList │
│ +0x020 WindowTitle                       │   │       ├─ ntdll.dll       │
│ +0x030 ImagePathName                     │   │       ├─ kernel32.dll    │
│ +0x040 CurrentDirectory                  │   │       └─ user32.dll      │
│ +0x070 CommandLine ⚠                     │   │ +0x18 InMemoryOrderModList │
│        └─> UNICODE_STRING                │   │ +0x20 InInitOrderModList │
│            Buffer ──────┐                │   │                          │
│                         │                │   │  ⚠ DLL Unlinking Target  │
│                         ▼                │   └──────────────────────────┘
│  ┌──────────────────────┐                │
│  │ Actual Command Line: │                │
│  │"certutil.exe -hashfile"│  ← This Section Is Manipulated!
│  └──────────────────────┘                │
└──────────────────────────────────────────┘
```

🎯 Attacker Objective:
ProcessParameters->CommandLine.Buffer

Laboratory Findings: System-Level Validation

To validate the telemetry desynchronization described in this study, controlled laboratory tests were conducted across two distinct scenarios.The scenarios were designed to demonstrate how the same technique behaves visibly in analysis environments while remaining significantly quieter in real-world attack conditions.

Scenario 1: Visible Child Process (Demonstration Mode)

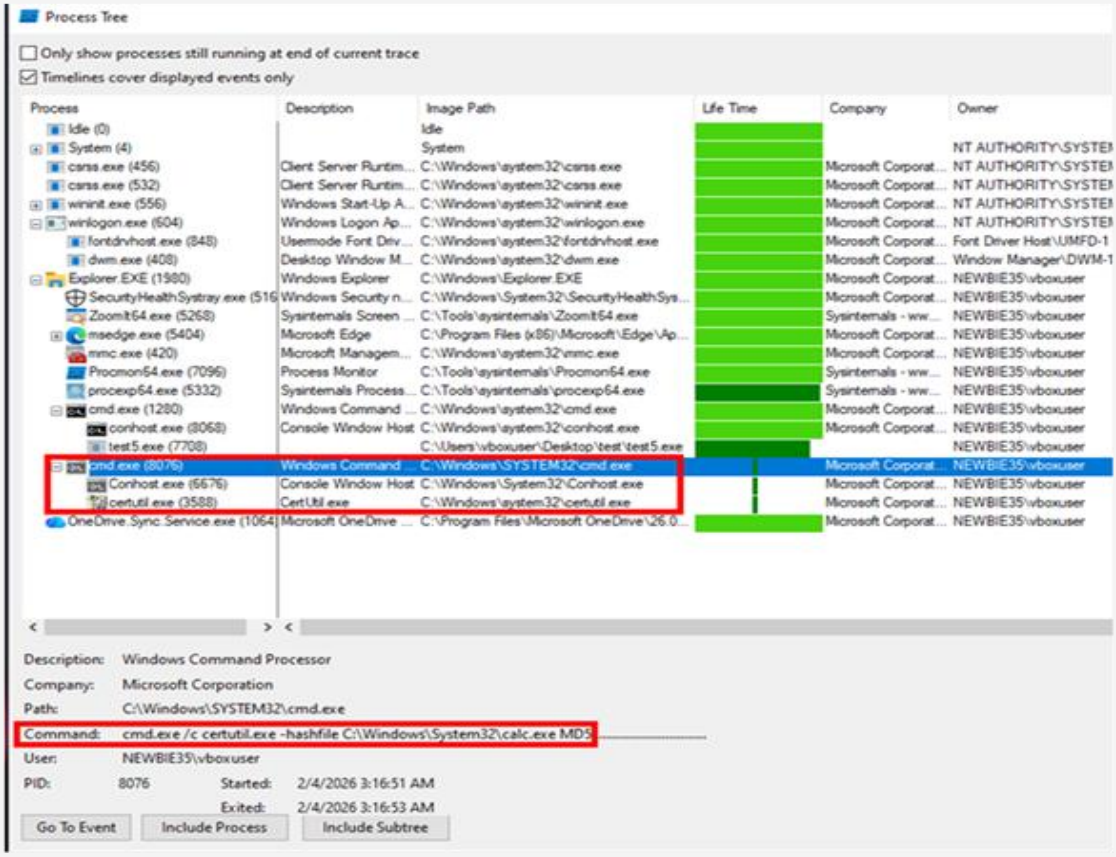
To make the impact of the technique explicitly observable, a scenario was intentionally constructed to spawn a child process. At first glance, the process tree and Sysmon Event ID 1 logs present a consistent and seemingly benign execution flow. However, closer inspection reveals a behavioral mismatch between the parent process command line and the actual activities performed by the child process.

In this scenario, malicious command execution remains observable within the process tree during EDR and sandbox analysis. The objective is to clearly and traceably demonstrate the fundamental effect of the technique.

Process Tree Analysis (Procmon)

Lab Observation: At this stage, no overtly suspicious activity is observed within the process tree. The command executed via **cmd.exe** is clearly visible, and it can be determined that **certutil.exe** retrieves the hash value of **calc.exe**.

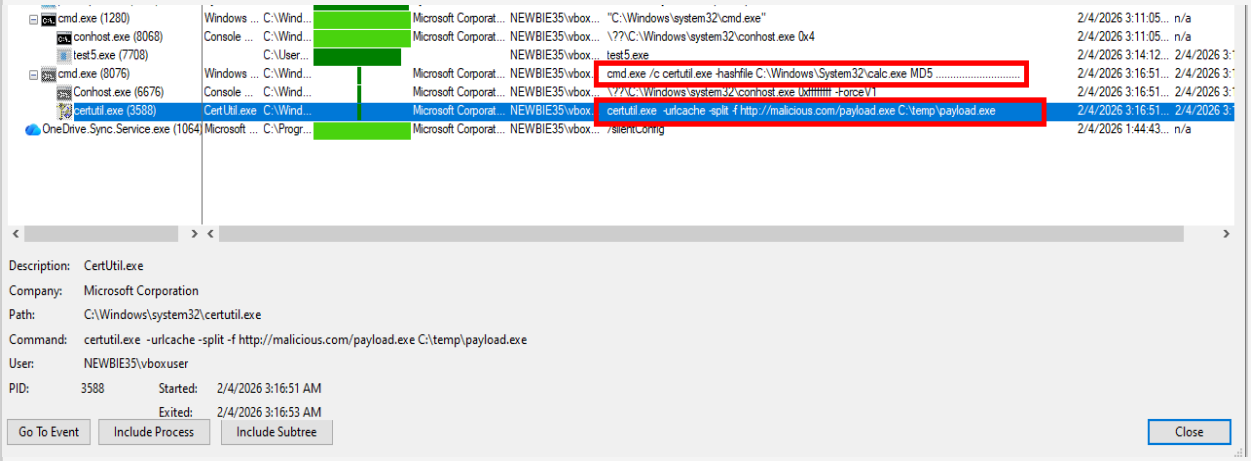
In this context, the appearance of **certutil.exe** as a child process is consistent and represents expected behavior from a process tree perspective.



Görsel 1 - Procmon

Process Tree Analysis (Procmon)

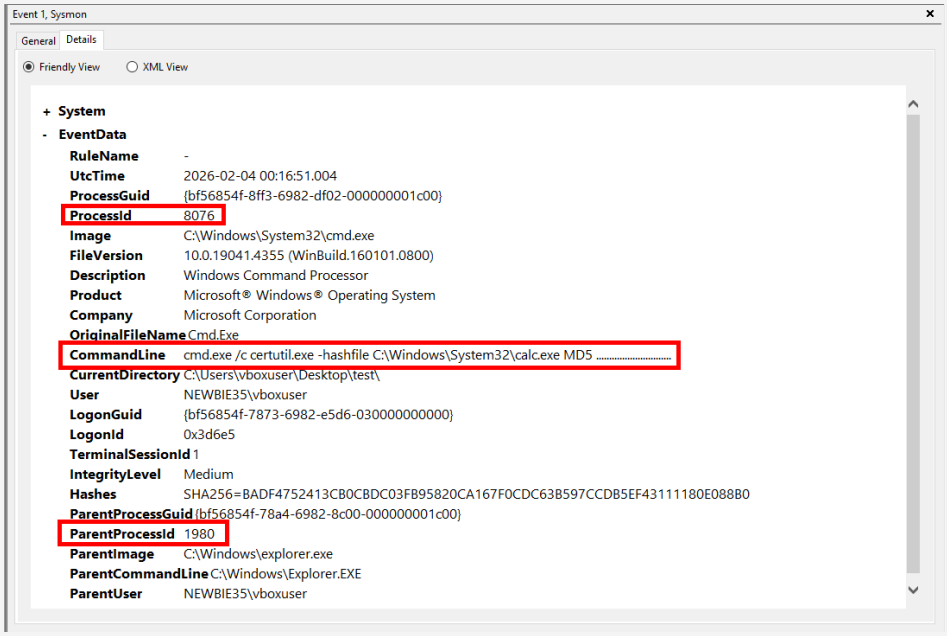
Lab Observation: Although the process tree appears consistent at first glance, there is a clear mismatch between the parent process command line and the actions performed by the child process. This visibility is the result of the scenario being intentionally designed to create a child process.



Görsel 2 - Procmon

Sysmon Event ID 1 – Process Creation

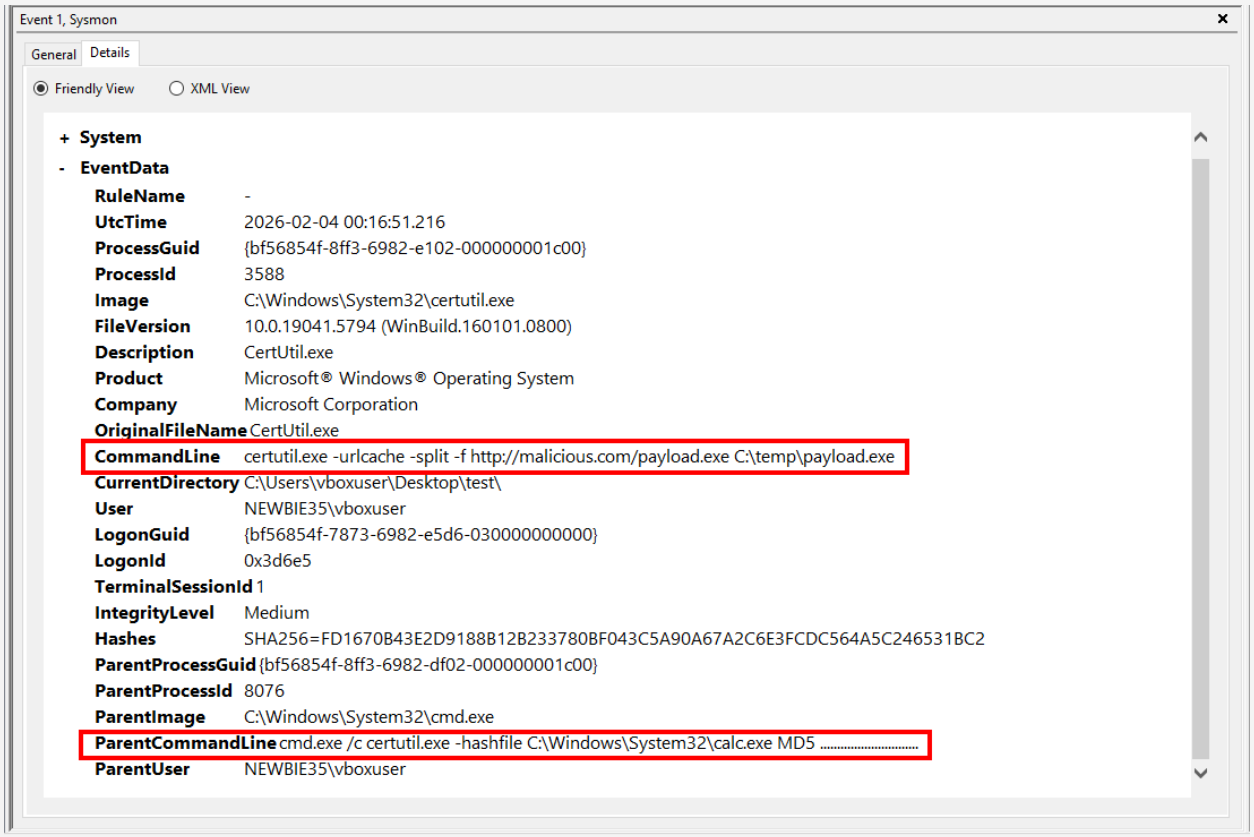
Lab Observation: Sysmon logs exhibit normal and benign behavior. The logs show only that a file hash is calculated using **certutil.exe**.



Görsel 3 - Sysmon log

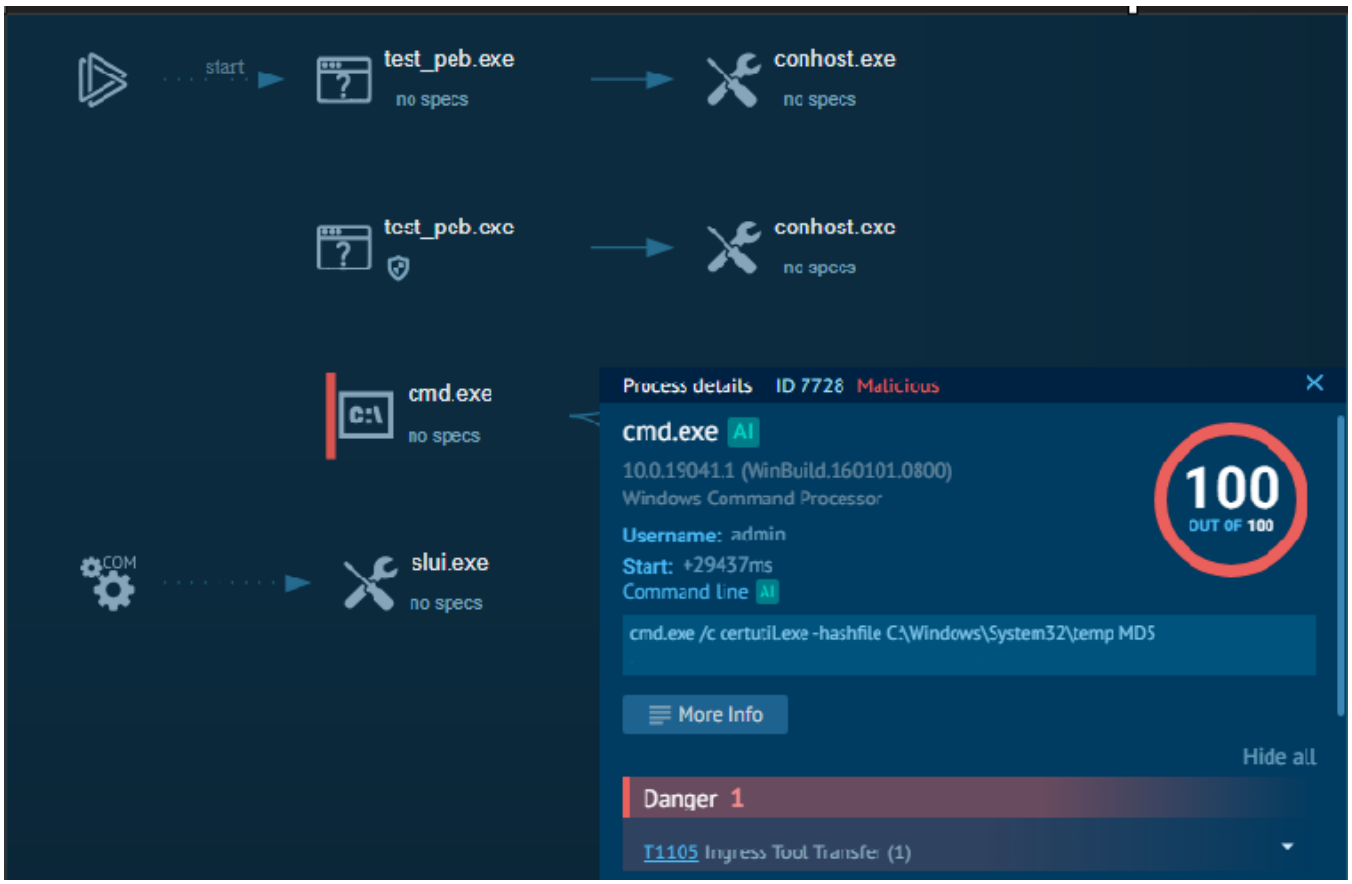
Sysmon Event ID 1 – Process Creation

Lab Observation: The activity was intentionally executed as a separate child process, which triggered an additional Sysmon event. While this event makes the background execution visible, there is a clear mismatch between the parent process command line and the actual execution.

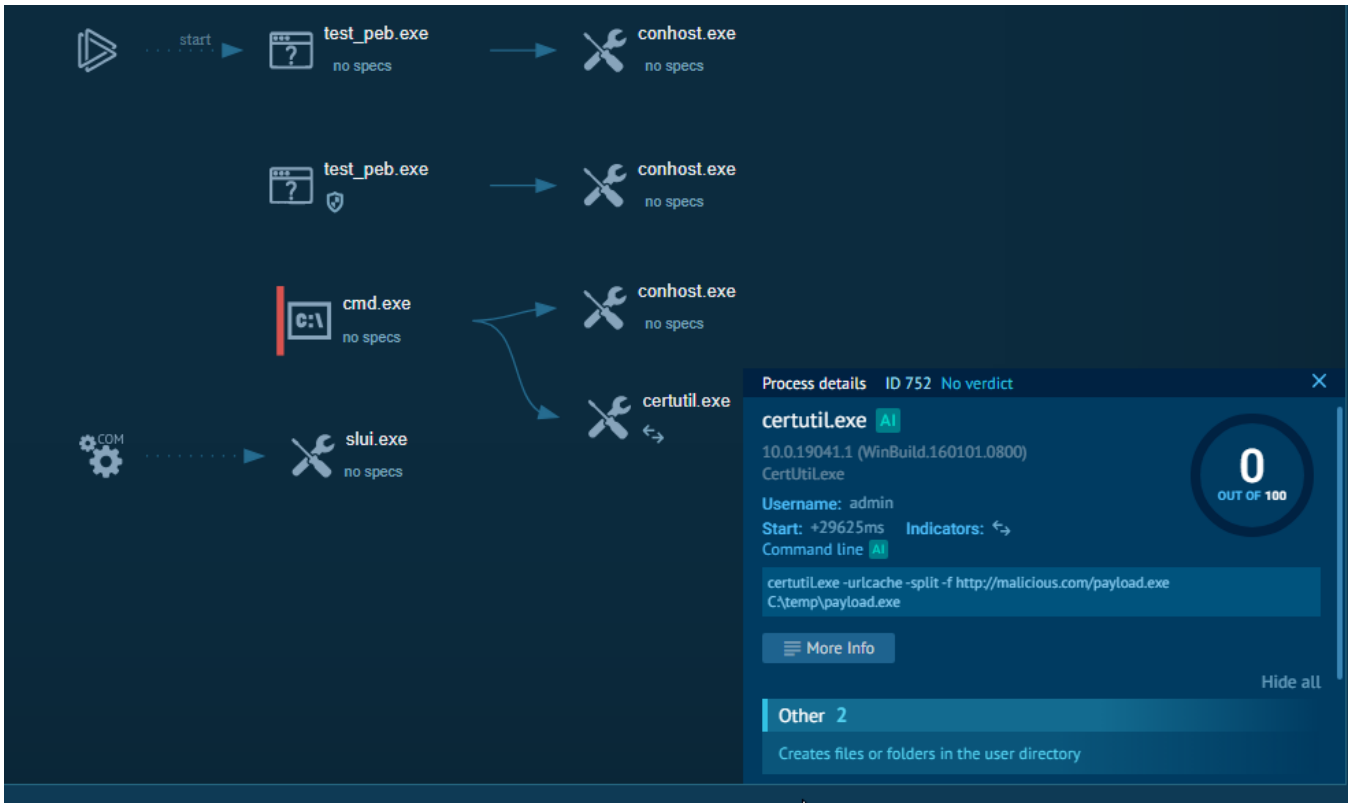


Görsel 4 - Sysmon log

ANYRUN



Görsel 5 - AnyRun



Görsel 6 - AnyRun

Scenario 2: Hidden Execution (Real Attack)

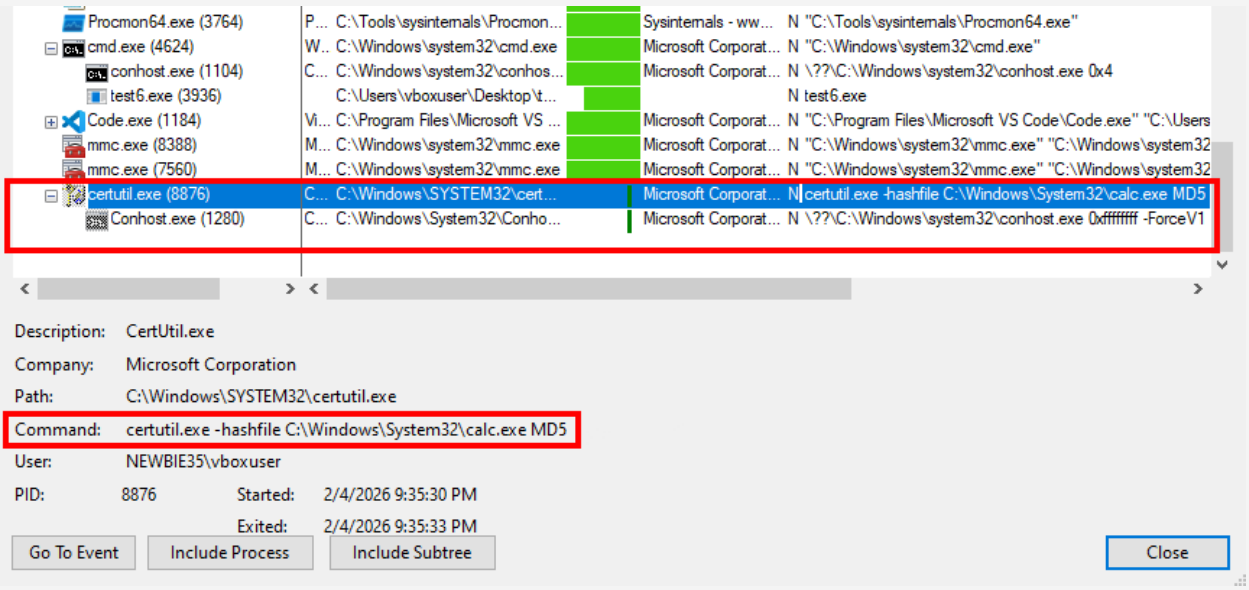
In this scenario, no additional child processes were created; the execution occurred within the existing process. While Procmon shows no traces of malicious activity in the process tree, Sysmon produced only a single Event ID 1 record. Under the default Sysmon configuration, process creation telemetry alone is insufficient to reveal this execution.

This scenario clearly demonstrates why detection based solely on process creation telemetry alone is not sufficient to reveal this execution.

(Note: Different results may be observed if Kernel ETW is enabled or if additional telemetry, such as Sysmon Event ID 10/11, is collected.)

Process Tree Analysis (Procmon)

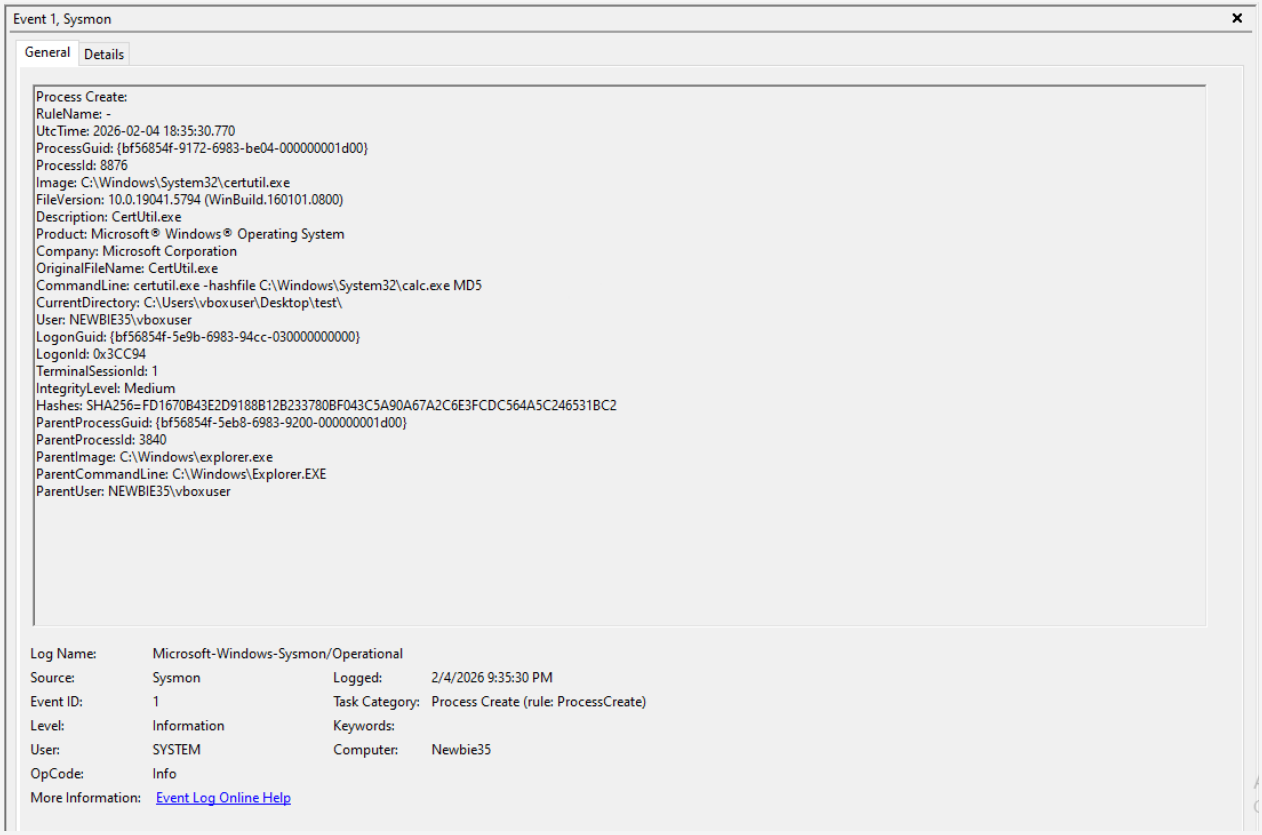
Lab Observation: Process tree analysis via Procmon revealed no visible child processes or commands associated with the background execution.



Görsel 7- Procmon

Sysmon Event ID 1 – Process Creation

Lab Observation: In this scenario, Sysmon telemetry generated only a single Event ID 1 record and did not provide any additional logs related to the executed activity.



Event 1, Sysmon

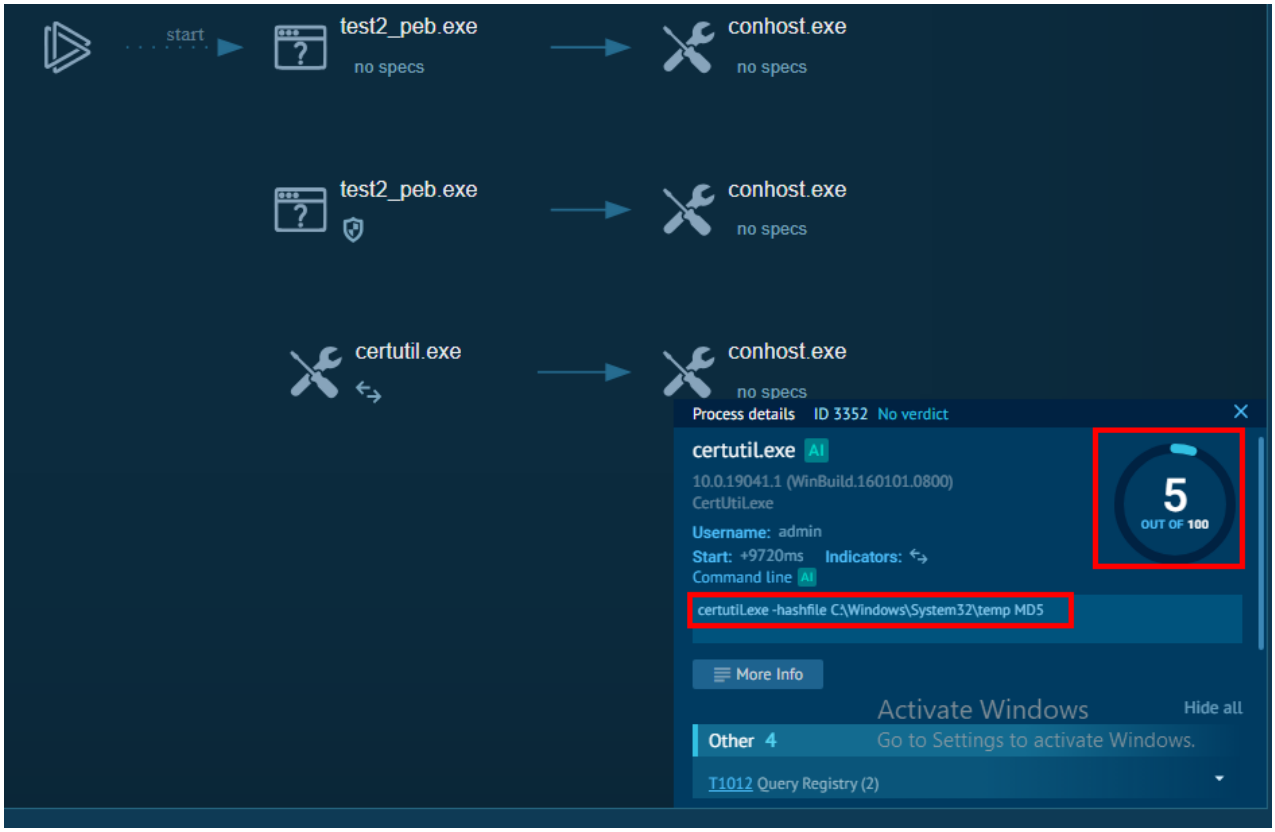
General | Details

```
Process Create:
RuleName: -
UtcTime: 2026-02-04 18:35:30.770
ProcessGuid: {bf56854f-9172-6983-be04-000000001d00}
ProcessId: 8876
Image: C:\Windows\System32\certutil.exe
FileVersion: 10.0.19041.5794 (WinBuild.160101.0800)
Description: CertUtil.exe
Product: Microsoft® Windows® Operating System
Company: Microsoft Corporation
OriginalFileName: CertUtil.exe
CommandLine: certutil.exe -hashfile C:\Windows\System32\calc.exe MD5
CurrentDirectory: C:\Users\vboxuser\Desktop\test\
User: NEWBIE35\vboxuser
LogonGuid: {bf56854f-5e9b-6983-94cc-030000000000}
LogonId: 0x3CC94
TerminalSessionId: 1
IntegrityLevel: Medium
Hashes: SHA256=FD1670B43E2D9188B12B233780BF043C5A90A67A2C6E3FCDC564A5C246531BC2
ParentProcessGuid: {bf56854f-5eb8-6983-9200-000000001d00}
ParentProcessId: 3840
ParentImage: C:\Windows\explorer.exe
ParentCommandLine: C:\Windows\Explorer.EXE
ParentUser: NEWBIE35\vboxuser
```

Log Name:	Microsoft-Windows-Sysmon/Operational		
Source:	Sysmon	Logged:	2/4/2026 9:35:30 PM
Event ID:	1	Task Category:	Process Create (rule: ProcessCreate)
Level:	Information	Keywords:	
User:	SYSTEM	Computer:	Newbie35
OpCode:	Info		
More Information:	Event Log Online Help		

Görsel 8 – Sysmon

AnyRun



Görsel 9 – AnyRun

As observed, the malicious command did not appear anywhere, and unlike our previous tests. **The activity executed as intended throughout the process.**

Görsel 10 – AnyRun

Why EDRs Sometimes Detect PEB Manipulation and Sometimes Don't?

The primary reason for the inconsistent behavior of EDRs in techniques like PEB manipulation is that process telemetry is collected from two distinct layers: kernel and user-mode.

At the kernel level, Windows passes the **PsCreateProcessNotifyInfo** structure to registered callbacks during process creation. The **CommandLine** field in this structure represents the actual command line at the moment the process is created. If the EDR driver relies on this callback, the information is obtained independently of user-mode memory structures and remains unaffected even if the PEB is later modified.

At the user-mode level, the process tree, command line, and visualization data are typically enriched via the PEB or APIs that read from the PEB. Since this reading occurs after process creation, any modifications made by an attacker to the PEB are directly reflected in this layer.

> This distinction explains why an EDR can detect this technique in some environments while remaining completely silent in others

If an EDR compares the process creation-time command line obtained from the kernel with the command line read later in user-mode, any discrepancy for the same process generates a strong detection signal. However, in practice, many EDRs do not perform this correlation automatically, or they may store kernel data in the background without exposing it in the UI.

From a behavioral perspective**,** PEB manipulation inherently requires a process to write into the memory of another process. Such accesses, performed via **`NtWriteVirtualMemory`** or similar calls, can be observed in "process access" telemetry like **Sysmon Event ID 10**. When an EDR evaluates these events together with process relationships, timing, and LOLBin usage, it can generate detections based on PEB write activity.

However, if an EDR relies solely on user-mode command line data, ignores kernel-level creation-time information, or fails to correlate the two layers, **PEB manipulation can remain completely invisible without triggering any alerts**.

> In short, detecting the technique does not depend solely on product capabilities; it is determined by which layer the data is collected from, at what time, and how these layers are correlated

5. EDR Bypass Techniques and Defensive Layers

```
═══════════════════════════════════════════════════════════════
                    EDR BYPASS Matrix
═══════════════════════════════════════════════════════════════

┌──────────────────────────────────────────────────────────┐
│ TECHNIQUE1: PEB MANIPULATION                             │
├──────────────────────────────────────────────────────────┤
│ Target: ProcessParameters->CommandLine                   │
│                                                          │
│ Bypassed:                                                │
│   ✗ Sysmon Event ID 1 (historical log view)              │
│   ✗ User-mode process monitoring                         │
│   ✗ SIEM correlation rules (benign command observed)     │
│                                                          │
│ Not Bypassed:                                            │
│   ✓ Kernel callbacks (real-time validation)              │
│   ✓ Memory forensics (VAD tree analysis)                 │
│   ✓ Cross-validation (Sysmon vs live process)            │
│                                                          │
│ Observed In: APT41, Cobalt Strike, LockerGoga            │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│ TECHNIQUE2: DLL UNLINKING                                │
├──────────────────────────────────────────────────────────┤
│ Target: PEB.Ldr->InMemoryOrderModuleList                 │
│                                                          │
│ Method:                                                  │
│   1. Reflective DLL injection                            │
│   2. The injected DLL is removed PEB loader-linked lists:
│      - InLoadOrderModuleList                             │
│      - InMemoryOrderModuleList                           │
│      - InInitializationOrderModuleList                   │
│   3. user-mode tools no longer display the DLL.          │
│                                                          │
│ Not Bypassed:                                            │
│   ✗ Process Explorer (reads PEB.Ldr)                     │
│   ✗ Task Manager (module enumeration)                    │
│   ✗ User-mode monitoring tools                           │
│                                                          │
│ Not Bypassed:                                            │
│   ✓ VAD tree analysis (kernel memory map)                │
│   ✓ Memory dumps (DLL remains in memory)                 │
│   ✓ Sysmon Event ID 7 (ImageLoad - creation time)        │
│                                                          │
│ Observed In: Cobalt Strike Beacon, APT29 WellMess        │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────────────┐
│ TEKNİK 3: CALL STACK SPOOFING                                        │
├──────────────────────────────────────────────────────────────────────┤
│ Target: Call stack integrity                                         │
│                                                                      │
│ Concept:                                                             │
│   1. Fake stack frames are introduced.                               │
│   2. Return address edirected to a legitimate modüle                 │
│   3. EDR Call stack originate from a trusted source                  │
│                                                                      │
│ Call Stack Inspection:                                               │
│   ┌──────────────────────────────────────┐                           │
│   │ VirtualAllocEx() called              │                           │
│   │  ↓ Stack trace:                      │                           │
│   │   ├─ ntdll.dll!NtAllocateVirtualMem  │ ✓ Legitimate              │
│   │   ├─ kernel32.dll!VirtualAllocEx     │ ✓ Legitimate              │
│   │   └─ [spoofed frame]                 │ ● Manipulated             │
│   │                                      │                           │
│   │ EDR : ALLOWED (appears legitimate)   │                           │
│   └──────────────────────────────────────┘                           │
│                                                                      │
│ Bypass Edilen:                                                       │
│   ✗ user-mode Call stack                                             │
│   ✗ Behavioral analysis (appears legitimate)                         │
│                                                                      │
│ Not Bypassed:                                                        │
│   ✓ Kernel-level stack validation                                    │
│   ✓ Return address consistency checks                                │
│   ✓ User vs kernel mode call stack comparison                        │
│                                                                      │
│ Observed In: APT41 DodgeBox, Cobalt Strike 4.5+                      │
└──────────────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────────────┐
│ TECHNIQUE 4: EDR SILENCER (WFP Filters)                              │
├──────────────────────────────────────────────────────────────────────┤
│ Objective: Disable EDR telemetry                                     │
│                                                                      │
│ Windows Filtering Platform (WFP) Abuse:                              │
│   1. WFP filter added                                                │
│   2. EDR network callbacks are blocked                               │
│   3. Process/thread creation otifications are dropped                │
│   4. File system events are filtered                                 │
│                                                                      │
│ Impact:                                                              │
│   ┌──────────────────────────────────────┐                           │
│   │ BEFORE:                              │                           │
│   │ ✓ Process created → EDR alerted      │                           │
│   │ ✓ Network connection → EDR logged    │                           │
│   │ ✓ File modified → EDR detected       │                           │
│   └──────────────────────────────────────┘                           │
│                                                                      │
│   ┌──────────────────────────────────────┐                           │
│   │ AFTER   (WFP filter active):         │                           │
│   │ ✗ Process created → EDR BLIND        │                           │
│   │ ✗ Network connection → EDR BLIND     │                           │
│   │ ✗ File modified → EDR BLIND          │                           │
│   └──────────────────────────────────────┘                           │
│                                                                      │
│ Not Bypassed:                                                        │
│   ✓ Kernel driver self-protection                                    │
│   ✓ Telemetry redundancy (multi-source collection)                   │
│   ✓ WFP filter audit (Event ID 5157)                                 │
│                                                                      │
│ Observed In: EDRSilencer (2024), Targeted ransomware                 │
└──────────────────────────────────────────────────────────────────────┘
```

Detection: Logs Are Not Wrong, But Incomplete

In techniques like PEB manipulation, the core problem is not missing logs. Logs are generated, they reach the SIEM, and in many cases appear “benign.” The issue is that these logs represent different moments of the same reality. Information recorded at process creation, metadata read during runtime, and events reflecting actual behavior do not align on the same timeline. The detection gap emerges precisely here.

Why Single-Source Detection Fails?

Most process analyses rely on the assumption: “Information captured at process creation remains valid during process execution.” This is not true. In Windows, some data is written by the kernel at creation time, while other data is read from user-mode memory during runtime. PEB manipulation targets this second category. Therefore, Sysmon, EDR interfaces, and other user-mode tools may report conflicting results not due to a software bug, but due to architectural differences.

The Real Signal: Inconsistency

Detecting these techniques is not just about looking for a “suspicious event.” The true signal lies in inconsistencies across sources. A process starting with a benign command line, long-lived LOLBin behavior, unexpected in-process accesses, and the semantic gap between creation-time and runtime information individually harmless become meaningful when analyzed together.

Key Question: " Is this process actually behaving as it reports?"

Why Is ETW Positioned Separately?

Kernel-based ETW telemetry observes process creation before user-mode manipulations, capturing parent relationships and timing in a more raw form. For this reason, techniques like PEB manipulation and parent spoofing may appear consistent in Sysmon or reasonable in the EDR interface, yet show as out-of-scope on the ETW side. ETW alone is not sufficient, but it provides a reference point that is harder to tamper with.

Behavior > Metadata

Command line, parent process, or image path alone are not reliable indicators; instead, more meaningful questions include: why did this process access another process’s memory, why did it generate network traffic, and why did it invoke an API it normally wouldn’t? Logs capturing in-process interactions, such as Sysmon Event ID 10, provide broader coverage than telemetry collected solely at process creation.

Section Takeaways:

- Logs do not lie, but they speak with incomplete context.
- PEB manipulation does not erase logs—it **distorts perception**.
- Effective detection requires **relationship and correlation analysis**, not a single event.
- Kernel and user-mode data are meaningful **only when analyzed together**, not separately.

Defense Architecture: Focus on Layers, Not a Single Log

Attacks of this type cannot be detected through a single log; adversaries deliberately rely on defenders trusting just one source. Effective defense depends not only on individual events but on layered analysis and correlation.

1- Process Creation (Ground Truth)

- Kernel ETW
- Process notify callbacks

→ *How did this process actually start?*

2- Snapshot Logs

- Sysmon Event ID 1
- EDR creation events

→ *What was reported to us at that moment?*

3- Runtime Behavior

- Sysmon Event ID 10
- API calls
- Process injection signals

→ *What did this process do while running?*

4- Context Analysis

- Parent-child relationships
- LOLBin chains
- Time correlation

→ *Does this behavior make sense?*

5- Live Verification

- EDR live response
- Process Explorer
- Memory snapshot

→ *What do I observe right now?*

6□ Cross-Source Verification

- Kernel ↔ User
- Log ↔ Memory
- Event ↔ Behavior

→ *Do these sources tell the same story?*

⚠ **Critical**: Even if one or more of these layers are missing, the system continues to operate; the attacker, however, exploits precisely these gaps.

BlackCat (ALPHV) Ransomware-as-a-Service

Target: Healthcare Provider - October 2022

Impact:	8 hospital offline, HIPAA breach, $22M hasar
Detection Time:	Immediate (users reported, systems down)
EDR Alarm:	0 (EDR killed at Stage 3)
Downtime:	11 gün (critical systems), 2.1M patient records

Attack Chain

Stage	Technique	Bypassed	Details
Initial access	**Compromised RDP Brute Force**	• Failed login alerts bypass • Geo-anomaly detection bypass	Citrix Gateway brüte force Weak password, No MFA
Reconnaissan + PEB Manipulation	**Domain Enumeration CommandLine Spoofing**	• Sysmon benign cmdline • SIEM recon rules bypass	nltest, net commands Actual: nltest /dclist
LSASS Dump	**PEB + Call Stack Combo Attack**	• Event ID 10 logged • EDR LSASS alert bypass	rundll32 comsvcs.dll Benign cmdline + legitimatecall stack
EDR Killer	**BYOVD + WFP Filter**	• EDR TELEMETRY	RTCore64.sys vulnerable driver Kernel-level EDR terminate
Lateral Movement	**PSExec + SMB PEB every hop**	• Lateral movement (EDR blind)	Stolen credentials
Backup Destruction	**Shadow Copy Delete Veeam Targeting**	• Backup monitoring (EDR blind)	vssadmin delete shadows /a l l Veeam backup server access
Ransomware	**BlackCat Rust Binary Multi-thread Encryption**	• File integrity • Mass encryption (EDR blind)	2.1M patient records DICOM files encrypted $10M Monero demand

APT41 - LockerGoga Ransomware

Target: Norsk Hydro - March 2019

Impact:	40+ factories shut down, $71M in damage
Detection Time:	72+ hours (manual investigation)
EDR Alarm:	0 (EDR Silencer active)
Downtime:	8 days from initial access to encryption

Attack Chain

Stage	Technique	Bypassed	Details
Initial access	**Spear-phishing + Weaponized Excel**	• Email Gateway • Sandbox	certutil.exe ile payload download + mshta.exe → HTA execution
PEB Manipulation	**CommandLine Spoofing NtWriteVirtualMemory**	• Sysmon Event ID 1 • EDR UI • SIEM correlation	Actual: certutil -urlcache C2 Displayed: certutil –hashfile MD5
DLL Unlinking	**Reflective DLL Injection PEB.Ldr Manipulation**	• Process Explorer • Task Manager • Module scanning	Cobalt Strike Beacon removed from PEB Ldr InMemoryOrderModuleList
Call Stack Spoofing	**Stack Frame Manipulation ROP Gadgets**	• EDR stack walking • Behavioral analysis • API call detection	VirtualAllocEx → ntdll.dll appearing as a legitimate call chain
Credential Harvesting	**LSASS Memory Dumping comsvcs.dll MiniDump**	• Sysmon Event ID 10 • LSASS access alerts • Credential theft	rundll32 + PEB manipulation using Microsoft-signed comsvcs.dll
Lateral Movement	**WMI + PsExec every host**	• Lateral movement • Admin activity • Network analytics	40+ server, 3,000+ endpoint Domain admin credentials
EDR Silencer	**WFP Filter Callback blocking**	• Process monitoring • Network callbacks • File events	EDR fully blind, telemetry reduced to 0%
Ransomware Deployment	**LockerGoga AES-256 Multi-threaded**	• File Integrity • Behavioral detection • Mass file alerts	Shadow copy deletion

APT29 - NOBELIUM Enterprise Breach

Target: Fortune 500 Technology Company - May 2021

impact:	Source code stolen, 3 months of C2 access
Detection Time:	3 months (Microsoft Threat Intelligence notification)
EDR Alarm:	(EDR Silencer was not used - not necessary)
Stolen Data:	2.3 TB of source code + internal documentation

Attack Chain

Stage	Technique	Bypassed	Details
Initial access	**Password Spray** **+ MFA Bypass**	• MFA (legacy auth) • Geo-location alerts • Login throttling	Azure AD account accessed via corporate VPN
PEB + PPID Spoofing	**Dual Spoofing** **Parent + CommandLine**	• Sysmon • EDR process tree • SIEM parent-child	Parent: cmd.exe → svchost.exe Cmdline: encoded → legitscript.ps1
Memory-Only Payload	**Fileless Execution** **Reflective DLL + Unlinking**	• File-based AV/EDR • Signature detection • YARA rules	Cobalt Strike Beacon Filess XOR + AES encrypted
DLL Unhookin + Call Stack	**EDR Hook Removal** **Syscall Direct**	• EDR API monitoring • Function hooking • Call validation	ntdll.dll unhook Direct kernel syscall + ROP chain
Persistence + C2	**WMI Event Subscription** **DNS Tunneling**	• Registry monitoring • Network C2 signatures • DPI/IDS	Named pipe: msagent Domain fronting CDN
Credential Theft	**LSASS Dump** **+ Token Manipulation**	• LSASS access detection • Credential theft • Privilege escalation	comsvcs.dll MiniDump SYSTEM + token duplicate SeDebugPrivilege
Data Exfiltration	**Steganography** **+ HTTPS Encryption**	• DLP solutions • Network traffic analysis • Exfiltration rules	2.3 TB over 3 months PNG metadata embedding Legitimate cloud storage

APT Defense Layer Bypass Matrix

Defense Layer	APT41	APT29	BlackCat	Bypass Technique
Email Gateway	✓	-	-	----------
Sandbox Analysis	✓	-	-	Delayed execution
Sysmon Event ID 1	✓	✓	✓	**PEB manipulation**
EDR Process Tree	✓	✓	✓	**PEB + PPID spoofing**
SIEM Korelasyon	✓	✓	✓	**PEB Benign görünüm**
Process Explorer	✓	✓	-	**PEB DLL unlinking**
Task Manager	✓	✓	-	**PEB Module enumeration bypass**
EDR Stack Walking	✓	✓	✓	**Call stack spoofing**
Behavioral Analysis	✓	✓	✓	**Meşru API origin**
LSASS Access Detection	✓	✓	✓	**Benign cmdline + stack**
Lateral Movement Detection	✓	✓	✓	Stolen admin creds
Network C2 Signatures	✓	✓	-	DNS tunneling / Domain fronting
File Integrity Monitoring	✓	-	✓	EDR silencer
DLP Solutions	-	✓	-	Steganography
All EDR Telemetry	✓	-	✓	WFP Filter / BYOVD

Conclusion

The key takeaway from this study is that the issue does not stem from a single technique. PEB manipulation, Sysmon's limitations, or certutil abuse alone are not the "root problem." **The true issue lies in treating logs as absolute truth**.

On Windows, some telemetry sources are fixed at the kernel level during process creation, while others are read from user-mode memory during runtime. This difference creates critical blind spots in the defense when timing gaps occur. As a result, different tools may present different realities for the same process.

> **Common Thread Across Scenarios:** Logs are not wrong—they are incomplete. Behavior that appears "benign" when viewed from a single source can clearly appear malicious once contextual information is expanded.

Key Takeaways:

- Process creation logs do not capture the full lifecycle of a process
- User-mode views may appear correct, but they do not necessarily reflect reality
- Kernel telemetry is more resilient, yet it cannot serve as the sole decision point
- Memory analysis provides the most reliable reference, but it represents only a snapshot in time.

Therefore, the question is no longer "**Are the logs correct?**".

The real question is: ***"At what time and under what conditions are these logs accurate?"***

Modern attacks are not primarily about hiding they are about distorting perception and scope.

Effective defense relies not on more rules, but on greater correlation, timing awareness, and multi-source analysis.

Key Takeaways

- ✓ A single telemetry source is never sufficient.
- ✓ Process creation logs, such as those from Sysmon, do not represent the full lifecycle of a process.
- ✓ Timing gaps create a concealment window for attackers.
- ✓ Processes should be evaluated as chains of events, not isolated occurrences.
- ✓ Behavioral analysis has surpassed signature and IOC-based detection
- ✓ Real-time observation complements, rather than replaces, historical log analysis

The techniques demonstrated in this study are not hypothetical, they are part of modern attack chains. PEB manipulation targets traditional assumptions of log trust and deliberately misleads single-source detection. Therefore, effective defense is not about writing more rules, but about building detection architectures capable of capturing timing gaps and telemetry inconsistencies.

Sources and References

- Velociraptor: ETW Part 2 - Process Parent Spoofing (2021)
- Trail of Bits: What child is this? Process Reparenting in Microsoft Windows (2022)
- Trail of Bits: ETW internals for security research and forensics (2023)
- Elastic Security Labs: Kernel ETW is the best ETW (2024)
- Huntress: I Wish I Was a Little Bit Taller - Dealing with Imperfection in Intrusions
- Microsoft: Process Environment Block (PEB) structure
- Microsoft: PsSetCreateProcessNotifyRoutineEx2 / PS_CREATE_NOTIFY_INFO
- SANS ISC: Malicious Process Environment Block Manipulation